

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A Coding

challenge

Code of Conduct:

I Suganthan Kumaresan (192424105) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Suganthan K

Annexure A
Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate\_segment(length\_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma\_log.txt`.
4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1ST QUESTION : STAR TOPOLOGY DIAGRAM GENERATOR

AIM

To write a Python program to generate a star topology diagram with multiple hosts connected to a central switch and display the Cat6 UTP cable length for each connection.

ALGORITHM

1. Start the program.
2. Read the number of hosts from the user.
3. Create a central switch named SW1.
4. Generate host names such as Host1, Host2, etc.
5. Assign a Cat6 UTP cable length to each host.
6. Display the switch, hosts, and cable connections.
7. Display the cable length for each connection.
8. Stop the program.

CODE :

```
def generate_star_topology(n):
    switch_name = "SW1"    print("\n===== STAR
TOPOLOGY =====\n")    print("
[SW1 -
Switch]")    print("
/ | \")    for i in range(1, n + 1):
host = f'Host{i}'    cable_length = 10 + (i * 5)    print(f"
{host:<10} --- Cat6 UTP --- {cable_length} m")
print("\nSwitch:", switch_name)    print("Cable Type: Cat6 UTP")
print("Total Hosts:", n)    n = int(input("Enter number of hosts: "))
if n > 0:
    generate_star_topology(n)
else:
    print("Number of hosts must be greater than 0.")
```

OUTPUT:

```
Enter number of hosts: 7
```

```
===== STAR TOPOLOGY =====
```

```
          [SW1 - Switch]
          /   |   \
Host1    --- Cat6 UTP --- 15 m
Host2    --- Cat6 UTP --- 20 m
Host3    --- Cat6 UTP --- 25 m
Host4    --- Cat6 UTP --- 30 m
Host5    --- Cat6 UTP --- 35 m
Host6    --- Cat6 UTP --- 40 m
Host7    --- Cat6 UTP --- 45 m
```

```
Switch: SW1
```

```
Cable Type: Cat6 UTP
```

```
Total Hosts: 7
```

RESULT

The Python program successfully generated a star topology diagram showing multiple hosts connected to a central switch along with the corresponding Cat6 UTP cable lengths.

2nd QUESTION : UTP SEGMENT LENGTH VALIDATION

AIM

To write a Python function `validate_segment(length_m)` to validate whether a UTP cable segment is within the 100-meter IEEE 802.3 limit.

ALGORITHM

1. Start the program.
2. Define the function `validate_segment(length_m)`.
3. Accept the UTP cable length in meters.
4. Check whether the length is less than or equal to 100 meters.
5. If the length is within the limit, return True.
6. If the length exceeds 100 meters, display an error message and return False.
7. Test the function with at least three different cable lengths.
8. Display the results.
9. Stop the program.

CODE :

```
def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        print(f'Error: Segment length {length_m} m exceeds the 100 m IEEE 802.3 limit.')
```

```

        return False
    print("Test Case 1:")
    result = validate_segment(50)
    print("Result:", result)
    print("\nTest Case 2:")
    result = validate_segment(100)
    print("Result:", result)
    print("\nTest Case 3:")
    result = validate_segment(120)
    print("Result:", result)

```

OUTPUT :

```

----- RESTART: D:/CH/Exp2.py -----
Test Case 1:
Result: True

Test Case 2:
Result: True

Test Case 3:
Error: Segment length 120 m exceeds the 100 m IEEE 802.3 limit.
Result: False

```

RESULT

The Python function successfully validated UTP cable lengths. Segments of 100 m or less returned True, while segments exceeding 100 m returned False with an appropriate error message.

3rd QUESTION : CSMA/CD SIMULATION WITH 4 HOSTS

AIM

To write a Python program to simulate CSMA/CD in a star topology with four hosts and record collision detection events and back-off intervals in a log file.

ALGORITHM

1. Start the program.
2. Create four hosts: Host1, Host2, Host3, and Host4.
3. Randomly select hosts that attempt to transmit.
4. Check whether multiple hosts transmit simultaneously.
5. If multiple hosts transmit, detect a collision.
6. Generate a random back-off interval.
7. Record the collision and back-off information in csma_log.txt.
8. If only one host transmits, record a successful transmission.
9. Repeat the simulation for multiple attempts.

10. Stop the program.

CODE :

```
import random
import time
LOG_FILE = "csma_log.txt"
hosts = ["Host1", "Host2", "Host3", "Host4"]
with open(LOG_FILE, "w") as log:
    log.write("===== CSMA/CD SIMULATION =====\n")
    for attempt in range(1, 11):
        transmitting_hosts = []
        for host in hosts:
            if random.random() < 0.4:
                transmitting_hosts.append(host)
        log.write(f"\nAttempt {attempt}\n")
        log.write(f"Transmitting Hosts: {transmitting_hosts}\n")
        if len(transmitting_hosts) > 1:
            log.write("COLLISION DETECTED!\n")
            k = min(attempt, 10)
            backoff_slots = random.randint(0, (2 ** k) - 1)
            log.write(f"Back-off interval: {backoff_slots} time slot(s)\n")
        )
        print(f"Collision: {transmitting_hosts} | "
              f"Back-off: {backoff_slots} slot(s)"
              )
        elif len(transmitting_hosts) == 1:
            log.write(f"Successful transmission by {transmitting_hosts[0]}\n")
        )
        print(f"Successful transmission: {transmitting_hosts[0]}")
        )
    else:
        log.write("No host transmitted.\n")
        print("No transmission.")
        time.sleep(0.2)
        log.write("\n===== SIMULATION COMPLETE =====\n")
        print(f"\nSimulation completed.")
        print(f"Log saved to {LOG_FILE}")
```

OUTPUT :

```
Successful transmission: Host2
Collision: ['Host2', 'Host3'] | Back-off: 0 slot(s)
Collision: ['Host1', 'Host2'] | Back-off: 3 slot(s)
Successful transmission: Host3
Collision: ['Host1', 'Host2'] | Back-off: 18 slot(s)
Successful transmission: Host3
Collision: ['Host1', 'Host3', 'Host4'] | Back-off: 42 slot(s)
Successful transmission: Host4
Successful transmission: Host4
Collision: ['Host1', 'Host2', 'Host3'] | Back-off: 179 slot(s)

Simulation completed.
Log saved to csma_log.txt
```

RESULT:

The Python program successfully simulated CSMA/CD, detected collisions between hosts, generated back-off intervals, and stored the collision events and transmission information in csma_log.txt.

4th QUESTION: CLIENT-SERVER STAR SWITCH COMMUNICATION

AIM

To develop a Python client-server program using the socket library to model communication between two hosts through a central star switch and display the switch's MAC address table after each frame transmission.

ALGORITHM

1. Start the switch/server program.
2. Create a TCP socket and bind it to an IP address and port.
3. Start listening for host connections.
4. Connect the client host to the switch.
5. Send the host ID, MAC address, and message to the switch.
6. Receive the frame at the switch.
7. Store the source host and MAC address in the MAC address table.
8. Display the updated MAC address table.
9. Send a forwarding confirmation to the client.
10. Repeat the process for the second host.
11. Stop the program.

CODE:

```
import socket
import threading
import time
SWITCH_MAC = "AA:BB:CC:DD:EE:FF"
mac_table = {}
def switch_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("127.0.0.1", 5000))
    server.listen(2)
    print("===== STAR SWITCH =====")
    print("Switch MAC:", SWITCH_MAC)
    print("Waiting for hosts...\n")
    for i in range(2):
        conn, address = server.accept()
        data = conn.recv(1024).decode()
        parts = data.split("|")
        host = parts[0]
        mac = parts[1]
        message = parts[2]
        mac_table[host] = mac
        print("\nFrame Received")
        print("Source Host :", host)
        print("Source MAC :", mac)
        print("Message   :", message)
        print("\nMAC Address Table")
        print("-----")
        print("Host\t\tMAC Address")
        print("-----")
        for h, m in mac_table.items():
            print(h, "\t\t", m)
        print("-----")
        conn.send("Frame forwarded successfully".encode())
        conn.close()
        server.close()
    server_thread = threading.Thread(target=switch_server)
    server_thread.start()
    time.sleep(1)
    client1 = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client1.connect(("127.0.0.1", 5000))
    frame1 = "Host1|00:11:22:33:44:01|Hello from Host1"
    client1.send(frame1.encode())
    response =
```



```

client1.recv(1024).decode() print("\nHost1
Response:") print(response) client1.close()
client2 = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client2.connect(("127.0.0.1", 5000)) frame2 =
"Host2|00:11:22:33:44:02|Hello from Host2"
client2.send(frame2.encode()) response =
client2.recv(1024).decode() print("\nHost2 Response:")
print(response) client2.close() server_thread.join()
print("\nCommunication completed successfully.")

```

OUTPUT:

```

===== STAR SWITCH =====
Switch MAC: AA:BB:CC:DD:EE:FF
Waiting for hosts...

Frame Received
Source Host : Host1
Source MAC  : 00:11:22:33:44:01
Message     : Hello from Host1

MAC Address Table
-----
Host          MAC Address
-----
Host1         00:11:22:33:44:01
-----

Host1 Response:
Frame forwarded successfully

Frame Received
Source Host : Host2
Source MAC  : 00:11:22:33:44:02
Message     : Hello from Host2

MAC Address Table
-----
Host          MAC Address
-----
Host1         00:11:22:33:44:01
Host2         00:11:22:33:44:02
-----

Host2 Response:
Frame forwarded successfully

Communication completed successfully.

```

RESULT:

The client-server program successfully demonstrated host-to-switch communication using Python sockets. The switch received frames, learned the MAC addresses of the hosts, and displayed the updated MAC address table after each transmission.

5th QUESTION: STARSWITCH CLASS

AIM

To implement a Python class StarSwitch that manages connected hosts and broadcasts frames to all ports except the source host.

ALGORITHM

1. Start the program.
2. Create the StarSwitch class.
3. Define the add_port(host_id) method to connect a host.
4. Define the remove_port(host_id) method to disconnect a host.
5. Define the broadcast(frame) method.
6. Identify the source host from the received frame.
7. Send the frame to all connected ports except the source port.
8. Log each frame delivery.
9. Display the broadcast results.
10. Stop the program.

CODE:

```
class StarSwitch:
    def __init__(self):
        self.ports = {}
    def add_port(self, host_id):
        """Add a host to the switch."""
        if host_id not in self.ports:
            self.ports[host_id] = []
            print(f'{host_id} connected to the switch.')
        else:
            print(f'{host_id} is already connected.')
    def remove_port(self, host_id):
        """Remove a host from the switch."""
        if host_id in self.ports:
            del self.ports[host_id]
            print(f'{host_id} disconnected from the switch.')
        else:
            print(f'{host_id} not found.')
    def broadcast(self, frame):
        """
        Broadcast frame to every port except
        the source host.
        Frame format:
        {
```

```

        'source': 'Host1',
        'message': 'Hello'
    }
    """

    source = frame["source"]
    message = frame["message"]
    print("\n===== BROADCAST =====")
    print("Source:", source)
    print("Message:", message)
    for host_id in self.ports:
        if host_id != source:
            self.ports[host_id].append(frame)
    print(
        f'Frame delivered from {source} '
        f'to {host_id}'
    )
    print("=====")
    switch = StarSwitch()
    switch.add_port("Host1")
    switch.add_port("Host2")
    switch.add_port("Host3")
    switch.add_port("Host4")
    frame = {
        "source": "Host1",
        "message": "Hello from Host1"
    }
    switch.broadcast(frame)
    print("\nRemoving Host4...")
    switch.remove_port("Host4")
    frame2 = {
        "source": "Host2",
        "message": "Hello from Host2"
    }
    switch.broadcast(frame2)

```

OUTPUT:

```
Host1 connected to the switch.
Host2 connected to the switch.
Host3 connected to the switch.
Host4 connected to the switch.

===== BROADCAST =====
Source: Host1
Message: Hello from Host1
Frame delivered from Host1 to Host2
Frame delivered from Host1 to Host3
Frame delivered from Host1 to Host4
=====

Removing Host4...
Host4 disconnected from the switch.

===== BROADCAST =====
Source: Host2
Message: Hello from Host2
Frame delivered from Host2 to Host1
Frame delivered from Host2 to Host3
=====
```

RESULT

The StarSwitch class was successfully implemented with add, remove, and broadcast operations. Frames were delivered to all connected hosts except the source host, and each delivery was successfully logged.